

Git y GitHub

Branches, Pull Requests, conflictos y estrategias de trabajo

Guion para exposicion oral - duracion aproximada: 10 minutos

Idea guia: usar el mismo ejemplo durante toda la exposicion: un equipo desarrolla una aplicacion y quiere agregar un login sin poner en riesgo la version estable.

5. Branches y trabajo en paralelo

Tiempo aproximado: 2 minutos

“Una de las funcionalidades mas importantes de Git son las ramas, tambien llamadas branches. Hasta ahora podemos imaginar que todo nuestro proyecto esta en una unica linea de trabajo llamada main. Main normalmente representa la version principal y estable del proyecto.”

“El problema aparece cuando queremos agregar una funcionalidad nueva. Por ejemplo, supongamos que tenemos una aplicacion funcionando correctamente y queremos desarrollar un sistema de inicio de sesion. Podriamos empezar a modificar directamente main, pero eso seria riesgoso: mientras programamos el login podemos cometer errores, dejar codigo incompleto o incluso romper algo que ya funcionaba.”

“Para evitarlo, creamos una rama nueva. Esa rama es como una linea de trabajo separada que parte del estado actual del proyecto.”

```
git switch -c feature/login
```

“A partir de ese momento podemos modificar archivos, probar cosas y hacer commits sin alterar directamente la rama principal. Por ejemplo:”

```
git add .  
git commit -m "Agrega formulario de inicio de sesion"
```

“Mientras tanto, otro integrante del equipo podria estar arreglando la validacion del correo electronico en otra rama, por ejemplo fix/validacion-email. Los dos pueden trabajar al mismo tiempo sin pisarse continuamente.”

```
main  
├── feature/login  
└── fix/validacion-email
```

“Esto es especialmente importante cuando trabajan varias personas en un mismo proyecto, porque cada cambio queda aislado y con un objetivo concreto. Cuando una funcionalidad esta terminada y probada, esa rama puede integrarse nuevamente con main mediante un merge.”

“En el grafico se ve justamente eso: las ramas salen de main, evolucionan por separado y despues vuelven a integrarse. Por eso las branches permiten trabajar en paralelo, aislar cambios y mantener main lo mas estable posible.”

Transicion: “Pero terminar una rama no significa que debamos incorporarla automaticamente a main. Antes normalmente aparece otro proceso muy importante: el Pull Request.”

6. Pull Request y Code Review

Tiempo aproximado: 2 minutos y 15 segundos

“Supongamos que terminamos feature/login y queremos integrar el nuevo login a main. Podriamos hacer un merge directamente, pero en un equipo normalmente queremos que alguien revise nuestros cambios antes de incorporarlos.”

“Para eso GitHub permite crear un Pull Request, normalmente abreviado como PR. Un Pull Request es una solicitud para integrar los cambios de una rama dentro de otra. En este caso estamos diciendo: estos son los cambios de feature/login y quiero incorporarlos a main.”

“Cuando creamos el Pull Request, GitHub nos muestra informacion muy util:”

- que archivos fueron modificados;
- que lineas de codigo fueron agregadas o eliminadas;
- que commits contiene la rama;
- y las diferencias entre nuestra rama y la rama de destino.

“Por ejemplo, podriamos ver que se agrego Login.tsx y que tambien se modifiko App.tsx para incluir la nueva pantalla de inicio de sesion.”

“Pero el Pull Request no sirve solamente para integrar codigo. Tambien funciona como un espacio de discusion. Ahi aparece el Code Review, o revision de codigo.”

“Otro integrante puede revisar nuestro trabajo y detectar cosas que nosotros no vimos.”

“La contraseña deberia validar un minimo de ocho caracteres.”

“Esta funcion esta repetida; podriamos reutilizar la que ya existe.”

“El desarrollador puede realizar esas modificaciones, hacer un nuevo commit y el Pull Request se actualiza automaticamente. Despues el revisor puede aprobar los cambios y recien entonces se hace el merge hacia main.”

```

Crear rama
↓
Desarrollar funcionalidad
↓
Hacer commits
↓
Push a GitHub
↓
Crear Pull Request
↓
Code Review
↓
Aprobacion
↓
Merge a main
  
```

“Esto mejora la calidad del código y evita que una sola persona decida por sí misma que su cambio ya está listo. Además, GitHub puede proteger main y exigir, por ejemplo, al menos una aprobación antes de permitir el merge.”

Transición: “Hasta ahora parece que integrar ramas siempre es automático. Sin embargo, existe una situación en la que Git necesita nuestra ayuda: los merge conflicts.”

7. Merge conflicts

Tiempo aproximado: 2 minutos

“Un merge conflict, o conflicto de fusión, ocurre cuando Git encuentra cambios incompatibles y no puede decidir automáticamente cuál debería conservar.”

“Veamos un ejemplo sencillo. Supongamos que en main tenemos este mensaje:”

```
const mensaje = "Bienvenido";
```

“Una persona crea una rama y lo modifica así:”

```
const mensaje = "Bienvenido al sistema";
```

“Mientras tanto, otra persona modifica exactamente esa misma línea y escribe:”

```
const mensaje = "Hola usuario";
```

“Cuando intentamos integrar las dos versiones, Git encuentra dos cambios diferentes sobre la misma parte del archivo y no sabe cuál es el correcto. Por eso marca el conflicto de esta manera:”

```
<<<<<< HEAD
const mensaje = "Bienvenido al sistema";
=====
const mensaje = "Hola usuario";
>>>>>> feature/mensaje
```

“La parte superior representa una versión y la inferior representa la otra. En ese momento una persona debe resolver manualmente el conflicto.”

“Podemos elegir una de las versiones o incluso combinar las dos. Por ejemplo:”

```
const mensaje = "Hola usuario, bienvenido al sistema";
```

“Después de editar el archivo y dejar solamente la versión correcta, indicamos a Git que el conflicto fue resuelto:”

```
git add .
git commit -m "Resuelve conflicto de mensaje"
```

“Algo importante es que resolver un conflicto no significa siempre quedarse con mi versión. La solución correcta depende del comportamiento que queremos conservar. Los conflictos son normales cuando varias personas trabajan sobre los mismos archivos; Git detecta el problema y deja la decisión en manos de los desarrolladores.”

Transición: “Ahora que sabemos cómo trabajar con ramas e integrarlas, falta una pregunta: ¿cómo decide un equipo que ramas crear y cómo utilizarlas? Para eso existen distintas estrategias de trabajo.”

8. GitFlow vs. GitHub Flow

Tiempo aproximado: 2 minutos y 30 segundos

“No todos los equipos utilizan las ramas de la misma manera. Existen distintas estrategias para organizar el desarrollo. Dos de las mas conocidas son GitFlow y GitHub Flow.”

“Primero tenemos GitFlow. Es un modelo mas estructurado y utiliza distintos tipos de ramas:”

```
main
develop
feature/*
release/*
hotfix/*
```

“Main contiene las versiones estables que llegan a produccion. Develop funciona como una rama donde se van integrando funcionalidades que todavia estan en desarrollo. Las ramas feature se usan para desarrollar funciones concretas, por ejemplo feature/login o feature/carrito.”

“Cuando estamos preparando una nueva version podemos utilizar una rama release, por ejemplo release/2.0. Y si aparece un error critico en produccion podemos crear una hotfix, por ejemplo hotfix/error-login.”

“GitFlow puede ser util en proyectos donde existen versiones o releases bien definidos y se necesita bastante control sobre las distintas etapas. Su desventaja es que tiene mas ramas y, por lo tanto, mas complejidad.”

“Por otro lado tenemos GitHub Flow, que es mucho mas simple. Normalmente partimos de main. Cuando queremos hacer un cambio creamos una rama corta, trabajamos en ella, subimos los cambios, abrimos un Pull Request, hacemos Code Review y, cuando esta aprobado, hacemos merge nuevamente hacia main.”

```
main
↓
crear branch
↓
hacer cambios
↓
Pull Request
↓
Code Review
↓
Merge
↓
main
```

“Despues normalmente esa rama se elimina porque ya cumplio su objetivo. La diferencia principal entre ambos modelos es la complejidad.”

GitFlow	GitHub Flow
Mas estructurado	Mas simple y liviano
main + develop + feature + release + hotfix	main + ramas cortas
Util para releases bien definidos	Util para integraciones frecuentes

Mas control, pero mas complejidad	Menos complejidad y ciclos mas cortos
-----------------------------------	---------------------------------------

“No podemos decir que uno sea siempre mejor que el otro. Depende del proyecto y del equipo. Si tenemos un software con versiones grandes y lanzamientos muy controlados, GitFlow puede resultar util. Si tenemos una aplicacion web donde hacemos cambios pequenos y frecuentes, GitHub Flow suele ser mas comodo.”

“Lo importante no es solamente conocer los comandos de Git, sino que todo el equipo tenga definido como va a utilizar Git.”

Cierre

“Entonces, si juntamos estos conceptos, podemos ver como Git permite que un equipo trabaje de manera organizada. Las branches permiten desarrollar cambios de manera aislada y trabajar en paralelo. Los Pull Requests permiten proponer esos cambios antes de incorporarlos. El Code Review permite que otros integrantes revisen el codigo. Los merge conflicts aparecen cuando Git encuentra cambios incompatibles y necesita que una persona decida como resolverlos. Y finalmente, estrategias como GitFlow o GitHub Flow establecen como un equipo organiza todo este proceso.”

“En conjunto, estas herramientas permiten que varias personas puedan modificar un mismo proyecto sin perder el control de los cambios.”

┃ *“Ahora vamos a ver como este flujo funciona directamente dentro de GitHub.”*